

8.3. GESTIÓN DE LA MEMORIA

En un sistema de monoprogamación, la memoria principal se divide en dos partes: una parte para el sistema operativo (el monitor residente) y otra parte para el programa que se está ejecutando. En un sistema multiprogramado, la parte de «usuario» de la memoria además debe subdividirse para dar cabida a los distintos procesos. La tarea de subdivisión la realiza dinámicamente el sistema operativo y se conoce como **gestión de memoria**.

Una gestión eficiente de la memoria es vital en un sistema multiprogramado. Si hay pocos procesos en memoria, puede ocurrir que todos los procesos estén esperando completar una E/S y el procesador permanecerá inactivo. En consecuencia, la memoria debe asignarse eficientemente para situar en memoria tantos procesos como sea posible.

INTERCAMBIO (SWAPPING)

Volviendo a la Figura 8.11, se han discutido tres tipos de colas: la cola a largo plazo para solicitar procesos nuevos, la cola a corto plazo con los procesos preparados para utilizar el procesador y las distintas colas de E/S de los procesos que no están preparados para usar el procesador. Recuérdese que la razón última de estos mecanismos era que las actividades de E/S son mucho más lentas que el cálculo y que por consiguiente el procesador en un sistema con uniprogamación está la mayor parte del tiempo parado.

Sin embargo, el esquema de la Figura 8.11 no resuelve el problema por completo. Es verdad que, en este caso, la memoria contiene múltiples procesos y que el procesador puede conmutar a otro proceso cuando el proceso en curso tenga que esperar. Pero el procesador es tan rápido en comparación con las E/S que puede ser frecuente que *todos* los procesos de la memoria estén esperando una E/S. Por eso, incluso con la multiprogamación, un procesador puede estar parado la mayor parte del tiempo.

¿Qué se puede hacer? La memoria principal podría ampliarse y así ser capaz de dar cabida a más procesos. Pero hay dos problemas en esta solución. Primero, incluso hoy día la memoria principal es cara. Segundo, la necesidad de memoria de los programas ha crecido tan rápido como ha caído el costo de la memoria. Por eso una memoria mayor origina procesos mayores, no más procesos.

Otra solución es el *intercambio (swapping)*, representado en la Figura 8.12. Tenemos una cola a largo plazo de solicitudes de proceso, usualmente almacenado en disco. Estas solicitudes se traen a memoria, una a una, a medida que hay espacio disponible. Conforme terminan, los procesos se sacan de la memoria principal. Ahora, podría ocurrir que ninguno de los procesos en la memoria principal esté en el estado preparado (por ejemplo, todos están esperando una operación de E/S). En lugar de permanecer parado, el procesador *intercambia* uno de esos procesos situándolo en el disco en una *cola intermedia*. Esta es una cola de procesos existentes que se han sacado temporalmente de memoria. El sistema operativo entonces trae otro proceso de la cola intermedia, o acepta una nueva petición de proceso de la cola de largo plazo. La ejecución continúa con el proceso recientemente activado.

El intercambio es de hecho una operación de E/S, y por consiguiente existe la posibilidad de empeorar el problema más que de solucionarlo. No obstante, puesto que la E/S en disco es generalmente la operación de E/S más rápida (comparada con la E/S en cinta o mediante impresora),

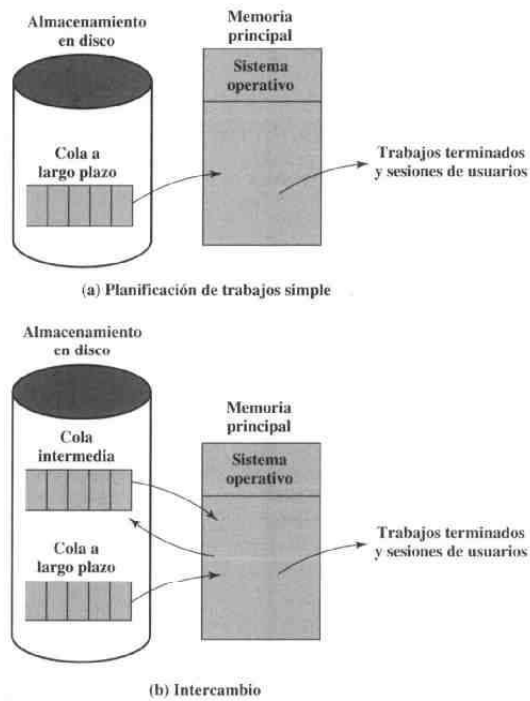


Figura 8.12. Uso del intercambio (*swapping*).

usualmente el intercambio mejora las prestaciones. Un esquema más sofisticado, que implica el uso de la memoria virtual, mejora las prestaciones con respecto al intercambio simple. Esto se discutirá en breve. Pero primero, debemos proporcionar los fundamentos explicando la definición de particiones y la paginación.

DEFINICIÓN DE PARTICIONES

El esquema más simple para definir particiones en la memoria disponible es utilizar *particiones de tamaño fijo*, como muestra la Figura 8.13. Observe que, aunque las particiones son de tamaño fijo, no todas tienen igual tamaño. Cuando un proceso se introduce en memoria, se sitúa en la partición disponible más pequeña que puede incluirlo.

Incluso con el uso de particiones de distintos tamaños, se desperdiciará memoria. En la mayoría de los casos, un proceso no necesitará exactamente la memoria que proporciona una partición. Por ejemplo, un proceso que precise 3 MB de memoria se podría situar en la partición de 4 M de la Figura 8.13b, desperdiciando 1 M que podrían utilizarse para otro proceso.

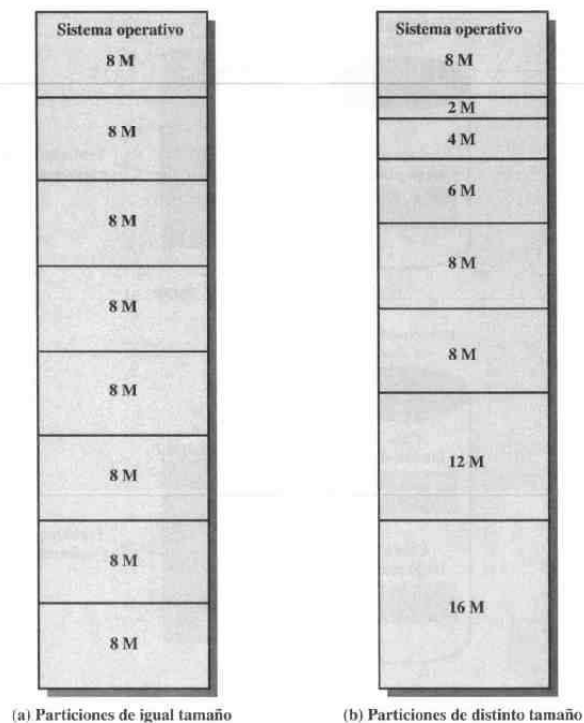


Figura 8.13. Ejemplo de particiones fijas de una memoria de 64 MB.

Una posibilidad más eficiente consiste en utilizar *particiones de tamaño variable*. Cuando un proceso se introduce en memoria, se le asigna exactamente la memoria que necesita y no más.

Ejemplo 8.2. En la Figura 8.14 se muestra un ejemplo utilizando 64 MB de memoria principal. Inicialmente, salvo por el sistema operativo, la memoria principal está vacía (a). Los primeros tres procesos se cargan, empezando por donde termina el sistema operativo y ocupando justo el espacio necesario para cada proceso (b,c,d). Esto deja un «hueco» al final de la memoria que es demasiado pequeño para un cuarto proceso. En cierto instante, ninguno de los procesos de memoria está preparado. El sistema operativo saca de memoria al proceso 2 (e), dejando espacio suficiente para cargar un nuevo proceso, el proceso 4 (f). Puesto que el proceso 4 es más pequeño que el proceso 2, se crea otro hueco pequeño. Posteriormente, se produce la situación en la que ninguno de los procesos que están en la memoria está preparado, excepto el proceso 2, que está disponible puesto que se encuentra en el estado Preparado-Suspendido. Como hay un espacio de memoria insuficiente para el proceso 2, el sistema operativo retira de memoria al proceso 1 (g) y vuelve a introducir al proceso 2 (h).

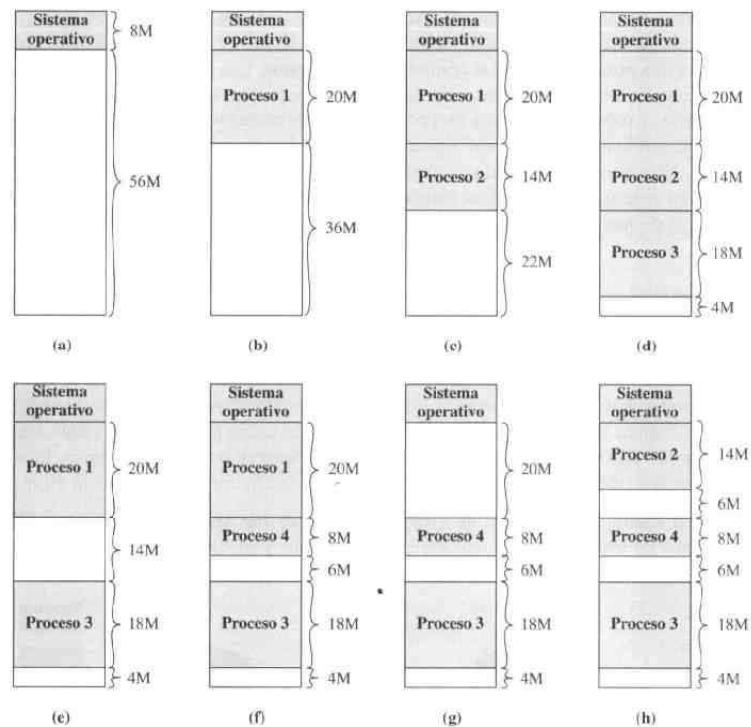


Figura 8.14. Efecto de la partición dinámica.

Como muestra este ejemplo, el método empieza bien, pero eventualmente puede llevar a situaciones en las que hay muchos huecos pequeños en memoria. A medida que pasa el tiempo, la memoria se fragmenta más y más, y empeora su utilización. Una técnica para solucionar este problema es la **compactación**: de vez en cuando, el sistema operativo desplaza los procesos en memoria para juntar toda la memoria libre en un bloque. Este es un procedimiento que consume parte del tiempo del procesador.

Antes de considerar formas de solucionar los problemas de la definición de particiones, debemos aclarar cierto extremo. Si el lector presta atención a la Figura 8.14 por un momento, resulta obvio que un proceso difícilmente se cargará en el mismo lugar de la memoria principal cada vez que se intercambia. Es más, si se utiliza compactación, un proceso puede desplazarse mientras se encuentra en memoria principal. La memoria del proceso está constituida por instrucciones y datos. Las instrucciones contendrán direcciones de posiciones de memoria de dos tipos:

- Direcciones de datos.
- Direcciones de instrucciones, usadas por las instrucciones de salto.

Pero estas direcciones no son fijas. Cambiarán cada vez que el proceso se intercambie. Para resolver este problema, se distingue entre direcciones lógicas y direcciones físicas. Una **dirección lógica** indica una posición relativa al comienzo del programa. Las instrucciones del programa contienen solo direcciones lógicas. Una **dirección física** es, por supuesto, la posición actual en la memoria principal. Cuando el procesador ejecuta un proceso, automáticamente convierte las direcciones lógicas en físicas sumando a cada dirección lógica la posición de comienzo actual del proceso, llamada **dirección base**. Este es otro ejemplo de un elemento hardware de la CPU diseñado para satisfacer las necesidades del sistema operativo. Las características exactas de este hardware dependen de la estrategia de gestión de memoria utilizada. Más adelante, en este mismo capítulo, veremos varios ejemplos.

PAGINACIÓN

Tanto las particiones de tamaño fijo como las de tamaño variable son ineficaces en el aprovechamiento de la memoria. Supóngase, no obstante, que la memoria se divide en trozos iguales de tamaño fijo y relativamente pequeño, y que cada proceso también se divide en pequeños trozos de tamaño fijo. Después los trozos de un programa, conocidos como **páginas**, se podrían asignar a los trozos de memoria disponibles, conocidos como **marcos (frames)**, o marcos de página. Entonces, el espacio de memoria desperdiciado por un proceso es, como mucho, una fracción de la última página.

La Figura 8.15 muestra un ejemplo del uso de las páginas y los marcos. En un instante dado, algunos de los marcos de memoria están ocupados y otros están libres. La lista de marcos libres es

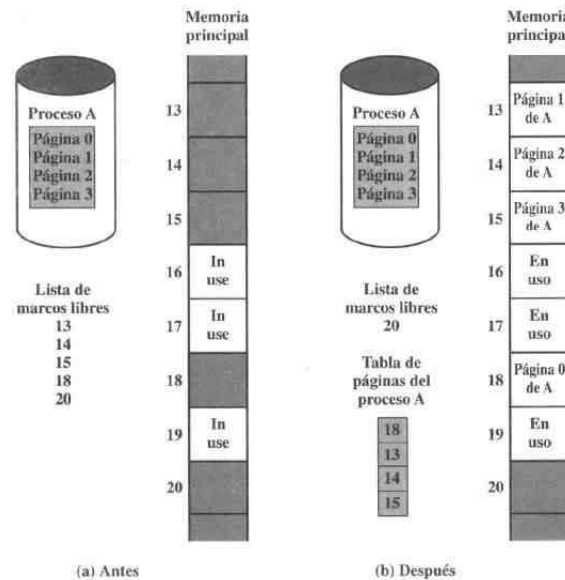


Figura 8.15. Asignación de marcos libres.

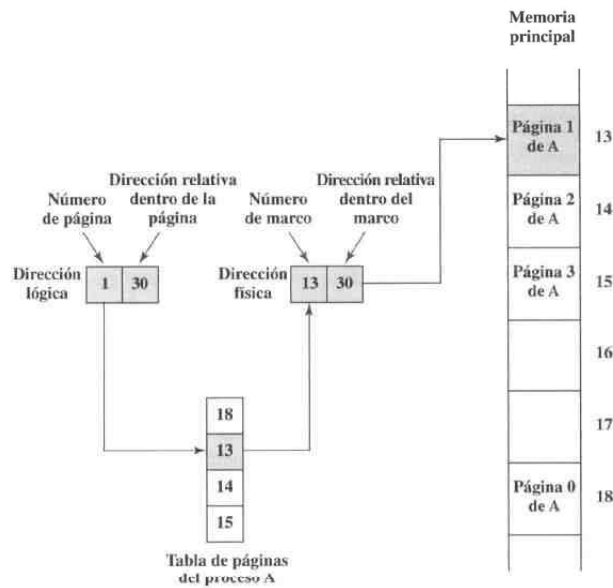


Figura 8.16. Direcciones lógicas y físicas.

gestionada por el sistema operativo. El Proceso A, almacenado en disco, consta de cuatro páginas. Cuando llega el momento de cargar este proceso, el sistema operativo encuentra cuatro marcos libres y carga las cuatro páginas del proceso A en cuatro marcos.

Supóngase ahora, como en este ejemplo, que no hay suficientes marcos contiguos sin utilizar para el proceso. (Hará esto que el sistema operativo no cargue A? La respuesta es no, porque de nuevo se utiliza una vez más el concepto de dirección lógica. Ya no es suficiente una única dirección de base. En cambio, el sistema operativo mantiene una **tabla de páginas** para cada proceso. La tabla de páginas indica el marco que aloja a cada página del proceso. Dentro del programa, cada dirección lógica está constituida por un número de página y una dirección relativa dentro de la página. Recuérdese que en el caso de particiones simples, una dirección lógica era la posición de una palabra en relación con el comienzo del programa; el procesador la traduce a una dirección física. Con la paginación, la traducción de dirección lógica a dirección física también la realiza el hardware del procesador. El procesador debe saber cómo acceder a la tabla de páginas del proceso en curso. A partir de la dirección lógica (número de página, dirección relativa), el procesador utiliza la tabla de páginas para generar la dirección física (número de marco, dirección relativa). Un ejemplo se muestra en la Figura 8.16.

Esta aproximación resuelve el problema anteriormente indicado. La memoria principal se divide en muchos marcos pequeños de igual tamaño. Cada proceso se divide en páginas del tamaño de los marcos; los procesos más pequeños necesitan menos páginas, los procesos mayores necesitan más. Cuando un proceso se lleva a memoria, sus páginas se cargan en los marcos disponibles y la tabla de páginas se actualiza.

MEMORIA VIRTUAL

Paginación por demanda. Con el uso de la paginación, se dispone de sistemas con multiprogramación verdaderamente efectivos. Es más, la sencilla táctica de dividir el proceso en páginas llevó al desarrollo de otro concepto decisivo: la memoria virtual.

Para entender la memoria virtual, debemos añadir una mejora al esquema de paginación discutido. Esta mejora es la **paginación por demanda**, que simplemente significa que cada página de un proceso se introduce en memoria solo cuando se necesita (es decir, cuando se solicita o demanda).

Considérese un proceso de tamaño elevado, consistente en un programa largo más un cierto número de matrices de datos. En un intervalo de tiempo corto, la ejecución puede confinarse a una pequeña sección del programa (por ejemplo una subrutina), y quizá solo se esté usando una o dos matrices de datos. Este es el principio de localidad, que se introdujo en el Apéndice 4A. Sería claramente un derroche cargar todas las páginas del proceso cuando solo se utilizarán unas pocas antes de que el proceso se suspenda. Podemos hacer un mejor uso de la memoria cargando solo unas pocas páginas. Después, si el programa salta a una instrucción de una página que no está en memoria principal, o si el programa hace referencia a un dato de una página que no está en memoria, se produce un **fallo de página**. Esto indica al sistema operativo que debe cargar la página deseada.

Así, en un instante dado, solo unas pocas páginas de un proceso están en memoria, y en consecuencia se pueden mantener en memoria más procesos. Además, se ahorra tiempo puesto que las páginas que no se utilizan no tienen que sufrir intercambios de almacenamiento. No obstante, el sistema operativo debe ser lo suficientemente ingenioso para manejar este esquema. Cuando se introduce una página en memoria, debe sacar otra fuera. Si saca una página justo en el momento en que va a empezar a utilizarse, tendrá que volver a introducirla en memoria casi inmediatamente; esto se denomina **reemplazo de página**. Si esto ocurre frecuentemente se produce una situación conocida como **hiperpaginación** (*thrashing*): el procesador pasa la mayor parte de su tiempo intercambiando páginas en lugar de ejecutar instrucciones. Las formas de evitar la hiperpaginación constituyeron una importante área de investigación en los setenta que dio lugar a una variedad de algoritmos complejos pero efectivos. En esencia, el sistema operativo intenta predecir, basándose en su historia reciente, qué páginas se utilizarán con menos probabilidad en el futuro próximo.

Con la paginación por demanda, no es necesario cargar el proceso entero en la memoria principal. Este hecho tiene una consecuencia importante: es posible que un proceso sea mayor que toda la memoria principal. Una de las restricciones más importantes de la programación ha sido vencida. Sin paginación por demanda, un programador debe tener en cuenta la memoria disponible. Si el programa que se está escribiendo es demasiado largo, el programador debe buscar formas de estructurar el programa en trozos que puedan cargarse uno a uno. Con demanda de página, ese trabajo se deja al sistema operativo y al hardware. En lo que al programador concierne, él o ella disponen de una cantidad de memoria enorme, el tamaño asociado al espacio en disco.

Puesto que un proceso se ejecuta solo si está en memoria principal, esta recibe el nombre de **memoria real**. Pero el programador o usuario percibe una memoria mucho mayor —la que hay disponible en disco. En consecuencia, esta última se denomina **memoria virtual**. La memoria virtual posibilita una multiprogramación muy efectiva y libera al usuario de las innecesarias y exigentes restricciones de memoria principal.

Estructura de la tabla de páginas. El mecanismo básico para leer una palabra de memoria implica la traducción, mediante una tabla de páginas, de una dirección virtual o lógica, consistente en un número de página y un desplazamiento a una dirección física, constituida por un número de marco y un desplazamiento. Puesto que la tabla de páginas tiene una longitud variable, dependiendo del tamaño del proceso, no es posible almacenarla en los registros. En su lugar, debe accederse a ella en memoria principal. La Figura 8.16 sugiere una implementación hardware de este esquema. Cuando un proceso determinado está ejecutándose, un registro contiene la dirección de inicio de la tabla de páginas de ese proceso. El número de página de una dirección virtual se utiliza como índice en la tabla para buscar el correspondiente número de marco. Este se combina con la parte de desplazamiento de la dirección virtual para construir la dirección real deseada.

En la mayoría de los sistemas, hay una tabla de páginas por proceso. Pero cada proceso puede ocupar una gran cantidad de memoria virtual. Por ejemplo, en la arquitectura VAX, cada proceso puede tener hasta $2^{31} = 2$ GB de memoria virtual. Utilizando páginas de $2^9 = 512$ bytes, eso significa que se necesitan tablas de páginas de 2^{22} elementos *por proceso*. Claramente, la cantidad de memoria dedicada solo a tablas de páginas podría ser inaceptablemente alta. Para solucionar este problema, la mayoría de los esquemas de memoria virtual almacenan las tablas de páginas en la memoria virtual en lugar de en la memoria real. Esto significa que la tabla de páginas también está sujeta a paginación igual que el resto de páginas. Cuando un proceso se está ejecutando, al menos una parte de su tabla de páginas, incluyendo el elemento correspondiente a la página actualmente en ejecución, debe estar en la memoria principal. Algunos procesadores hacen uso de un esquema de dos niveles para organizar las tablas de páginas grandes. En este esquema, hay una página de directorio en la que cada elemento apunta a una tabla de páginas. Así, si la longitud de la página de directorio es X , y si la longitud máxima de una tabla de páginas es Y , un proceso puede estar constituido por hasta $X \times Y$ páginas. Típicamente, la longitud máxima de una tabla de páginas se restringe al tamaño de una página. Veremos un ejemplo de esta aproximación de dos niveles más adelante en este mismo capítulo, cuando estudiemos el Pentium II.

Una aproximación alternativa al uso de tablas de páginas de uno o dos niveles es el uso de una estructura de *tabla de páginas invertida* (Figura 8.17). Variaciones de esta aproximación se utilizan en el PowerPC, el UltraSPARC y la arquitectura IA-64. Una implementación del sistema operativo Mach en el RT-PC también usa esta técnica.

En esta aproximación, la porción de la dirección virtual correspondiente al número de página se mapea en una tabla de dispersión (*hash*) mediante una función de dispersión sencilla². La tabla de dispersión incluye un puntero a una tabla de páginas invertida, que contiene los elementos de la tabla de páginas. Con esta estructura, hay un elemento en la tabla de dispersión y en la tabla de páginas invertida para cada página de memoria real en vez de para cada página de memoria virtual. Así, se necesita una porción fija de la memoria real para las tablas independientemente del número de procesos o páginas virtuales que se admitan. Puesto que más de una dirección virtual puede apuntar al mismo elemento de la página de dispersión, se utiliza una técnica de encadenamiento para solucionar

² Una función de dispersión asocia números comprendidos entre 0 y M con números entre 0 y N , donde $M > N$. La salida de la función de dispersión se utiliza como índice en la tabla de dispersión. Puesto que más de una entrada se asocia a la misma salida, es posible que un elemento de entrada apunte a una posición de la tabla de dispersión que está ocupada. En ese caso, el nuevo elemento debe pasar a otra posición de la tabla de mezcla. Usualmente, el nuevo elemento se sitúa en la primera posición vacía que se encuentra, y se establece un puntero desde la posición original para encadenar juntas las posiciones que se van ocupando. Véase [STAL98] para una discusión más detallada de las tablas de dispersión.

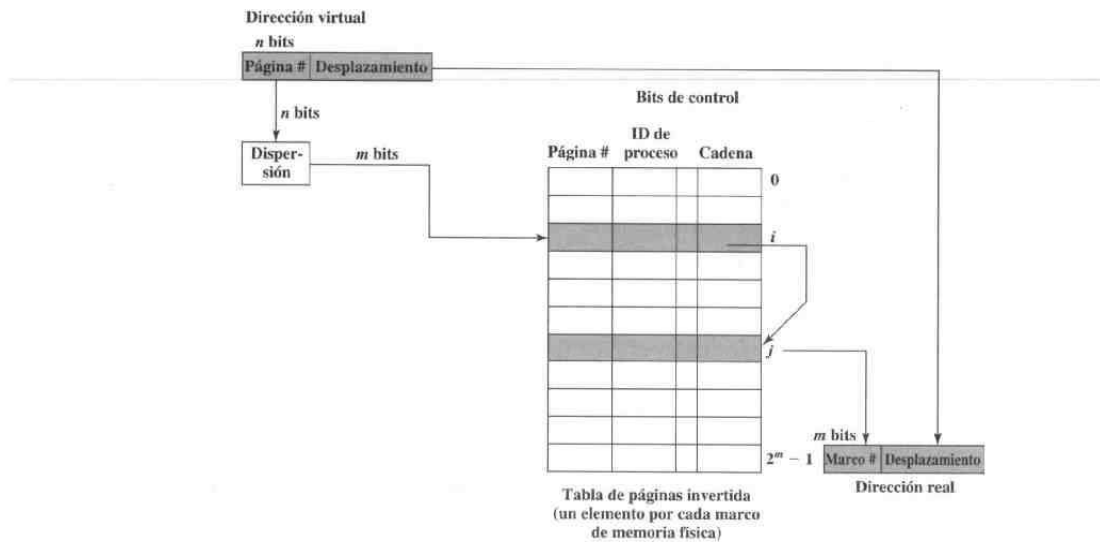


Figura 8.17. Estructura de la tabla de páginas invertida.

este problema. La técnica de dispersión da lugar a cadenas usualmente cortas —con uno o dos elementos. La estructura de la tabla de páginas se denomina estructura invertida debido a que indexa elementos de la tabla de páginas a través del número de marco en lugar de mediante el número de página virtual.

BUFFER DE TRADUCCIÓN ANTICIPADA (TRANSLATION LOOKASIDE BUFFER)

En principio, toda referencia a memoria virtual puede ocasionar dos accesos a la memoria física: uno para captar el elemento de la tabla de páginas apropiada, y otro para captar el dato deseado. Como consecuencia, un esquema de memoria virtual directo tendría el efecto de duplicar el tiempo de acceso a memoria. Para resolver este problema, la mayoría de los esquemas de memoria virtual hacen uso de una caché especial para los elementos de la tabla de páginas, llamada usualmente buffer de traducción anticipada (TLB, *Translation Lookaside Buffer*). Este buffer funciona de la misma manera que una memoria caché y contiene aquellos elementos de la tabla de páginas a los que se ha accedido más recientemente. La Figura 8.18 es un diagrama de flujo que muestra el uso del TLB. Por el principio de localidad, la mayoría de las referencias a memoria corresponderán a posiciones de las páginas recientemente usadas. Por eso, la mayoría de las referencias implican a elementos de la tabla de páginas incluidas en el TLB. Estudios del TLB de VAX muestran que este esquema puede mejorar significativamente las prestaciones [CLAR85, SATY81].

Observe que el mecanismo de memoria virtual debe interactuar con el sistema de caché (no con la caché que implementa el TLB, sino con la caché de la memoria principal). Esto se ilustra en la

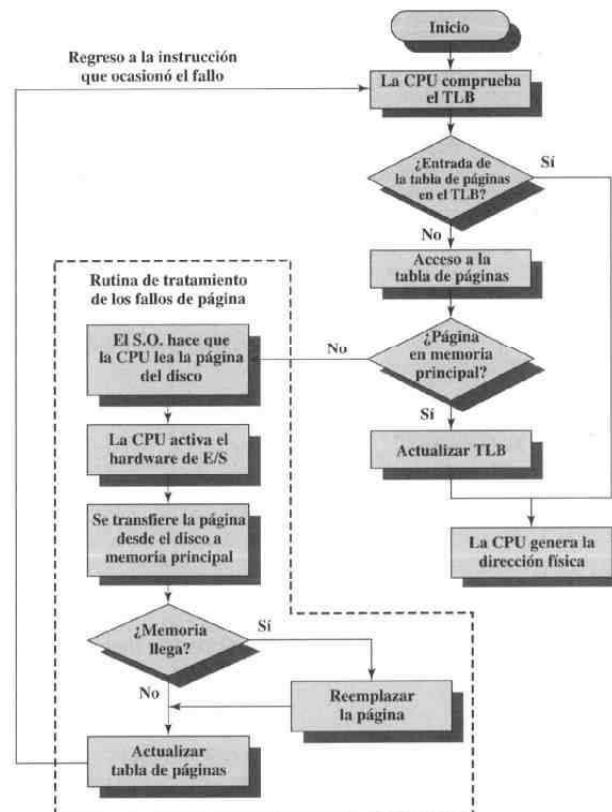


Figura 8.18. Funcionamiento de la paginación y del buffer de traducción anticipada (TLB) [FURH87].

Figura 8.19. Una dirección virtual estará generalmente en la forma de número de página más desplazamiento. Primero, el sistema de memoria consulta el TLB para comprobar si hay coincidencia con algún elemento de la tabla de páginas incluido en él. Si es así, se genera la dirección real (física) combinando el número de marco con el desplazamiento. Si no, se accede al elemento correspondiente de la tabla de páginas. Una vez que se ha generado la dirección real, constituida por una marca y los bits restantes (véase la Figura 4.17), se consulta la caché para ver si el bloque que contiene la palabra está presente. Si es así, se envía al procesador. Si no, se busca la palabra en memoria principal.

El lector puede apreciar la complejidad del hardware del procesador implicado en una simple referencia a memoria. La dirección virtual es traducida a una dirección real. Esto implica una referencia a la tabla de páginas, que puede estar en el TLB, en memoria principal o en disco. La palabra referenciada puede estar en caché, en memoria principal o en disco. En este último caso, la página

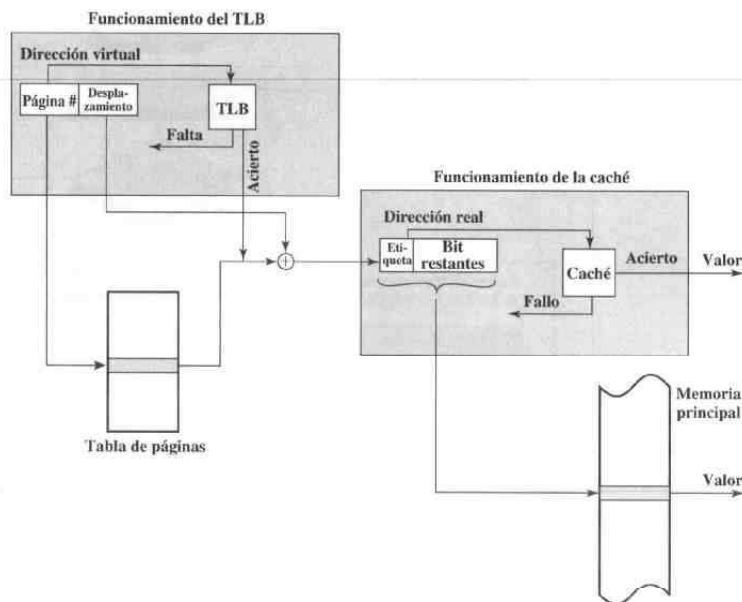


Figura 8.19. Buffer de traducción anticipada y funcionamiento de la caché.

que contiene a la palabra debe cargarse en la memoria principal y su bloque pasar a la caché. Además, el elemento de la tabla de páginas correspondiente a esa página debe actualizarse.

SEGMENTACIÓN

Hay otra forma en la que puede subdividirse la memoria direccionable, conocida como *segmentación*. Mientras que la paginación es invisible para el programador y sirve para proporcionar al programador un espacio de direcciones mayor, la segmentación es usualmente visible para el programador y proporciona una forma conveniente de organizar los programas y los datos, para asociar los privilegios y los atributos de protección con las instrucciones y los datos.

La segmentación permite que el programador vea la memoria constituida por múltiples espacios de direcciones o segmentos. Los segmentos tienen un tamaño variable, dinámico. Usualmente, el programador o el sistema operativo asignarán programas y datos a segmentos distintos. Puede haber segmentos de programa distintos para varios tipos de programas y también distintos segmentos de datos. Se pueden asignar a cada segmento derechos de acceso y uso. Las referencias a memoria se realizan mediante direcciones constituidas por un número de segmento y un desplazamiento.

Esta organización tiene ciertas ventajas para el programador frente a un espacio de direcciones no segmentado:

1. Simplifica la gestión de estructuras crecientes de datos. Si el programador no conoce a priori el tamaño que puede llegar a tener una estructura de datos particular, no es necesario que lo presuponga. A la estructura de datos se le asigna su propio segmento y el sistema operativo lo expandirá o lo reducirá según sea necesario.
2. Permite modificar los programas y recompilarlos independientemente, sin que sea necesario volver a enlazar y cargar el conjunto entero de programas. De nuevo, esto se consigue utilizando varios segmentos.
3. Permite que varios procesos compartan segmentos. Un programador puede situar un programa correspondiente a una utilidad o una tabla de datos de interés en un segmento que puede ser direccionado por otros procesos.
4. Se facilita la protección. Puesto que un segmento se construye para contener un conjunto de programas o datos bien definido, el programador o el administrador del sistema puede asignar privilegios de acceso de forma adecuada.

Estas ventajas no se tienen con la paginación, que es invisible para el programador. Por otra parte, hemos visto que la paginación proporciona una forma eficiente de gestionar la memoria. Para combinar las ventajas de ambas, algunos sistemas están equipados con el hardware y el software del sistema operativo que permite las dos.

8.4. GESTIÓN DE MEMORIA EN EL PENTIUM II Y EN EL POWERPC

HARDWARE DE GESTIÓN DE MEMORIA EN EL PENTIUM II

Desde la introducción de las arquitecturas de 32 bits, los microprocesadores han desarrollado esquemas de gestión de memoria sofisticados aprovechando la experiencia obtenida con los sistemas de medio y gran tamaño. En muchos casos, las versiones de los microprocesadores son superiores a sistemas de media y gran escala que les antecedieron. Puesto que los sistemas fueron desarrollados por los fabricantes del hardware de microprocesadores y debieran poder utilizarse con una cierta variedad de sistemas operativos, tienden bastante a ser de uso general. Un ejemplo representativo es el esquema utilizado por el Pentium II. El hardware de gestión de memoria del Pentium II es esencialmente el mismo que se usa en los procesadores 80386 y 80486 de Intel, con ciertas mejoras.

Espacios de direcciones. El Pentium II incluye hardware tanto para segmentación como para paginación. Ambos mecanismos se pueden desactivar, permitiendo elegir entre cuatro formas de ver la memoria:

- **Memoria no segmentada y no paginada:** en este caso, la dirección virtual es la misma que la dirección física. Esto es útil, por ejemplo, cuando se utiliza como controlador de baja complejidad y elevadas prestaciones.
- **Memoria paginada no segmentada:** la memoria se ve como un espacio de direcciones lineal paginado. La protección y la gestión de memoria se realiza via paginación. Esta es la forma preferida por ciertos sistemas operativos (por ejemplo, el UNIX de Berkeley).